

VoteCore

Today's development chat - project change log

Work area	Outcome
Visual interface	Civic Ledger design, contrast and fraud-log readability improvements
Backend security	Session validation, CSRF protection, rate limits, OTP controls, access scoping
Local operation	VS Code frontend/backend launch guidance, CORS and cached-script troubleshooting
Election data model	Organizer, party, position, per-post voting and audit-oriented schema planning

Scope

This document summarizes the work, decisions, troubleshooting, and database migration progress from today's VoteCore conversation. It is a project record, not a replacement for source control or a production deployment checklist.

1. User interface and accessibility work

- Refreshed the interface around a navy, paper, gold and civic-blue visual system.
- Corrected low-contrast text, including the Global Super Admin banner and fraud-log rows.
- Added dedicated fraud-log row styling so every log entry remains readable rather than only the highlighted alert.
- Improved profile layout behavior and restored a visible organization empty state: 'You have not joined any organizations yet.'
- Added a clear profile error state when the backend cannot be reached.

2. Authentication, sessions and security work

- Required the Flask secret key from environment configuration and hardened session-cookie defaults.
- Replaced predictable OTP generation with secrets-based random generation and limited OTP attempts.
- Added login attempt throttling and active-account validation.
- Added database-backed session validation and current-device / all-device logout support.
- Added CSRF token issuance after OTP verification, automatic frontend attachment for authenticated writes, and later corrected CORS preflight handling.
- Added profile read/update endpoints and access checks around organization and election actions.

3. Local runtime and email delivery troubleshooting

- Diagnosed that a Python runtime without Flask could not start the backend; the project virtual environment was verified as the correct runtime.
- Created a backend launcher and later restored the normal VS Code arrangement: frontend on port 5501 and backend on port 5000.
- Diagnosed stale browser JavaScript caused by temporary port changes; cache-busted script references so pages load current frontend code.
- Added clear user-facing messages when the backend is unavailable instead of silently redirecting or leaving empty sections.
- Validated SMTP configuration and Gmail App Password authentication. SMTP delivery was given a longer timeout and one retry for transient connection closures.
- Clarified that the backend terminal must remain running when using the project locally through VS Code.

4. Election-system design decisions

- Defined a hierarchy: Super Admin, Election Organizer, Party Leader, Candidate and Voter.
- Defined the strict neutrality rule: an Organizer cannot contest, create a party, or join a party in their assigned election.
- Defined an election lifecycle: organizer appointment, party formation, slate/ticket allocation, scrutiny/voting, and results declaration.
- Identified ballot secrecy as a core requirement: voter identity must be separated from the anonymous ballot record.
- Recommended post-by-post voting, explicit deadlines, voter-roll freeze, complaints/appeals, observer access, recount handling and provisional results.

5. Database-schema migration work

- Reviewed the existing Oracle 21c XE schema and identified that it supported basic elections but not election-specific parties, organizer appointments, posts, or per-post anonymous voting.
- Outlined new structures: PLATFORM_ROLE, PLATFORM_USER_ROLE, ORGANIZER_APPLICATION, ELECTION_POSITION, ELECTION_PARTY, PARTY_JOIN_REQUEST, ELECTION_PARTY_MEMBER, VOTER_POST_STATUS, ELECTION_PHASE_HISTORY, ELECTION_COMPLAINT, ELECTION_RESULT_DECLARATION and ELECTION_RECOUNT.
- Added election-scoped organizer, voter-roll-lock, result-status, candidate-party and candidate-position fields conceptually to the migration plan.
- Explained that final NOT NULL constraints must wait until old candidate data is migrated and Flask code creates party-linked candidates correctly.

Confirmed Oracle migration progress

Item	Status recorded today
Pre-checks	No duplicate emails found; existing statuses reviewed.
Organizer	Aditya (PID 2) appointed as Organizer for Election 1.
Positions	President and Vice President records exist for Election 1; existing candidates were mapped to position IDs.

Item	Status recorded today
Neutrality	Aditya's active President candidacy was withdrawn and retained with timestamp/reason for audit.
Deferred hardening	Final NOT NULL constraints and final party-position uniqueness remain intentionally deferred.

6. Important lessons from the Oracle SQL session

- SQL*Plus does not automatically understand placeholders such as :election_id. Replace them with real values, or explicitly declare bind variables first.
- Do not paste Markdown separators or explanatory lines into SQL*Plus. They are treated as commands and can cause SP2 errors.
- Enable SET SQLBLANKLINES ON before pasting multiline SQL containing blank lines.
- A unique constraint may automatically create its own index; ORA-01408 in this case indicated that a duplicate manual index was unnecessary.
- Do not apply NOT NULL constraints while migration rows still contain NULL values. Migrate and validate first.

7. Recommended next work

- Decide whether to reset the present election/test data or continue migrating it into parties and party tickets.
- If resetting, retain only the Super Admin, then have the Super Admin create new elections.
- Update the Flask backend and frontend so normal users can apply to become Organizer, while only the Super Admin can appoint one organizer per election.
- Implement party registration, Party Leader approval of members, post allocation, Organizer scrutiny, and party-linked candidate creation.
- Update voting to use VOTER_POST_STATUS and anonymous VOTE inserts inside one database transaction.
- Only after the application code is migrated should the deferred NOT NULL and final uniqueness constraints be enabled.

Current implementation note: the database structures can prepare the new election model, but the requested sign-in and organizer-application behavior requires matching Flask and frontend changes.